AMRITA VISHWA VIDYAPEETHAM – CHENNAI CAMPUS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
23CSE302 – COMPUTER NETWORKS

EXPERIMENT 2

INTRODUCTION TO SOCKET PROGRAMMING USING PYTHON

Name: Chetan Kumar G		Max Marks (10)
Roll No.: CH.SC.U4CSE24109	Section: CSE-B	
Date of Exp:02/07/2026	Date of Submission: 02/07/2026	

OBJECTIVES:

1. **Students will gain skill set in Socket Programming**
2. **Students will understand how a basic socket can be created using Python, resolve the hostname of a website to its IP address, and also establish a connection to the website's server**

A. IMPLEMENTING A BASIC CLIENT-SERVER COMMUNICATION IN PYTHON

AIM

To create a basic client-server communication setup using socket programming in Python, where the server listens for incoming connections and the client connects to the server to exchange a simple message.

SYSTEM AND SOFTWARE REQUIREMENTS

Software – Python Terminal

Libraries - socket

PROCEDURE

A. Server

1) Import Required Libraries:
   - Import the socket library.
2) Create a Socket:
   - Use the socket.socket() method to create a socket.
3) Bind the Socket to a Port:
   - Reserve an active free port on the computer (e.g., 12346).
   - Use the bind() method to bind the socket to the port.
4) Listen for Incoming Connections:
   - Use the listen() method to enable the server to accept connections. Specify the maximum number of connections (e.g., 5).
5) Accept and Handle Connections:
   - Use a while loop to keep the server running.
   - Use the accept() method to accept incoming connections.
   - Send a thank-you message to the connected client.
   - Close the connection.
   -

B. *Client*

1) Import Required Libraries:
   - Import the socket library.

2) Create a Socket:
   - Use the socket.socket() method to create a socket.
3) Connect to the Server:
   - Use the connect() method to connect to the server at the specified IP address (127.0.0.1) and port (12346).
   - Receive and Display Message:
   - Use the recv() method to receive a message from the server.
   - Print the received message.
   - Close the connection.

PYTHON CODE

A. Server

server.py > ...

```python
import socket
host = "0.0.0.0"
port = 12345
server = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
server.bind((host,port))
server.listen(1)
print("Waiting for Chetan2...")
conn,addr = server.accept()
print("connected to Chetan2")
while True:
    msg = conn.recv(1024).decode()
    if not msg:
        break
    print("Chetan2: ",msg)
    reply = input("Chetan: ")
    conn.send(reply.encode())
    if (reply.lower() == "thank you"):
        break
conn.close()
```

B. Client

client.py > ...

```python
import socket
ip="127.0.0.1"
port=12345
client=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
client.connect((ip,port))
while True:
    msg=input("chetan2:")
    client.send(msg.encode())
    reply=client.recv(1024).decode()
    if (reply.lower() == "thank you"):
        print("Connection closed by server.")
        break
    print("chetan:",reply)

```

OUTPUT

```
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet - 2> python server.py
Waiting for Chetan2...
connected to Chetan2
Chetan2:  hello
Chetan: how are you
Chetan2:  fine, what about you
Chetan: thank you
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet - 2>
```

```
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet - 2> python client.py
chetan2:hello
chetan: how are you
chetan2:fine, what about you
Connection closed by server.
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet - 2>
```

FIG 2. Client-Server Communication – short message

RESULT

The client-server communication was successfully established using Python socket programming. The server listened for incoming client connections, accepted the connection, sent the message "Thank you", and closed the connection successfully. The client connected to the server, received and displayed the message, and terminated the connection successfully.

INFERENCE

The experiment demonstrated that socket programming enables communication between a client and a server over a network. The server successfully listened for incoming connections, accepted a client request, sent a message, and closed the connection. The client connected to the server, received the message, and displayed it successfully. This verifies the basic working of TCP socket-based client-server communication in Python.

Lab Scenario: Basic Socket Programming and Client-Server Communication in Python
Scenario Title: ***"Employee Workstation Communication System"***
Background:
Your company, **TechBridge Solutions**, has recently moved into a new office building. The IT department wants to test basic internal communication between employee workstations using custom Python socket programs, before deploying more complex systems.
You have been assigned to build a **simple network communication prototype** using **Python socket programming**, where:

- Each **workstation** can act as a **client**.
- A central **server** runs on the manager's machine and listens for messages.
- Clients should connect to the server using the hostname and IP address of the server machine.
- Once connected, they should be able to send a short message like "Hello from Workstation 3" to the server.
- The server should print and acknowledge every message received.

Part A: Basic Socket Programming and Host Connection

1. Write a Python program to retrieve and print:
    - The hostname of the local machine.
    - The IP address of the local machine.

    CODE:

```
import socket

hostname = socket.gethostname()
ip_address = socket.gethostbyname(hostname)

print("Hostname:", hostname)
print("IP Address:", ip_address)
```

    OUTPUT:

```
PS C:\Users\chetan kumar g\OneDrive - A
Hostname: LAPTOP-HCB8QFVI
IP Address: 192.168.0.4
PS C:\Users\chetan kumar g\OneDrive - A
```

Q1: What is the hostname and IP address of your system?

```
PS C:\Users\chetan kumar g\OneDrive - A
Hostname: LAPTOP-HCB8QFVI
IP Address: 192.168.0.4
PS C:\Users\chetan kumar g\OneDrive - A
```

2. Write another Python program that:
    - Resolves the hostname of a given remote server (e.g., "www.google.com") to its IP address.

Q2: What is the IP address of www.google.com resolved by your code?

```python
import socket

hostname = "www.google.com"
ip_address = socket.gethostbyname(hostname)

print("Hostname:", hostname)
print("IP Address:", ip_address)
```

```
IP Address: 192.168.0.4
PS C:\Users\chetan kumar g\OneDrive
Hostname: www.google.com
IP Address: 142.251.151.119
PS C:\Users\chetan kumar g\OneDrive
```

Part B: Implementing Basic Client-Server Communication

3. **Server-Side Requirements:**
   - Create a Python socket server that:
     - Listens on a port (e.g., 12345)
     - Accepts connections from clients
     - Receives a message and prints it
     - Sends back an acknowledgment: "Message received"
4. **Client-Side Requirements:**
   - Create a client that:
     - Connects to the server using its IP and port
     - Sends a message (e.g., "Hello from Workstation X")
     - Receives the acknowledgment and prints it

Q3: Did the server receive the message from the client? Paste the output from both client and server terminals.

```python
import socket
host = "127.0.0.1"
port = 12345
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((host, port))
server.listen(5)
print("Server is listening on port", port)
while True:
    conn, addr = server.accept()
    print("Connected to:", addr)
    message = conn.recv(1024).decode()
    print("Client:", message)
    conn.send("Message received".encode())
    conn.close()
```

```python
import socket
host = "127.0.0.1"
port = 12345
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((host, port))
message = "Hello from Workstation 3"
client.send(message.encode())
reply = client.recv(1024).decode()
print("Server:", reply)
client.close()
```

```
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V Sem\Computer Networks\Lab Activity\Worksheet - 2> python bserver.py
Server is listening on port 12345
Connected to: ('127.0.0.1', 61774)
Client: Hello from Workstation 3
```

```
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V Sem\Computer Networks\Lab Activity\Worksheet - 2> python bclient.py
Server: Message received
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V Sem\Computer Networks\Lab Activity\Worksheet - 2>
```

Extension Task (Optional Challenge):
Modify the server to handle multiple clients using threading so that it can process messages from several workstations at once.
Q4: How did you implement multithreading? Did the server respond to all clients without blocking?

Multithreading was implemented using Python's threading module. Each time a client connected, the server created a separate thread to handle communication with that client. This allowed multiple clients to communicate with the server simultaneously without waiting for previous clients to disconnect.

Yes. The server responded to all connected clients independently without blocking other incoming connections, demonstrating concurrent client handling.

cserver.py > ...

```python
import socket
import threading
host = "127.0.0.1"
port = 12345
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((host, port))
server.listen(5)
print("Server is running...")
def handle_client(conn, addr):
    print("Connected:", addr)
    message = conn.recv(1024).decode()
    print(f"{addr}: {message}")
    conn.send("Message received".encode())
    conn.close()
while True:
    conn, addr = server.accept()
    thread = threading.Thread(target=handle_client, args=(conn, addr))
    thread.start()
```